

Machine Learning

Learn PyTorch by Example (4): Sequence Prediction with Recurrent Neural Networks (I)

Learn PyTorch by example, use Recurrent Neural Networks to predict the sine function

Jan 27, 2024 12 minute read

简体中文

Background

This is the fourth article in the "Learn PyTorch by Example" series. In the previous three articles:

- "Learn PyTorch by Example (1): PyTorch Basics and MNIST Handwritten Digit Recognition (I)"
- "Learn PyTorch by Example (2): Parameter Selection in MNIST Handwritten Digit Recognition (II)"
- "Learn PyTorch by Example (3): MNIST Handwritten Digit Recognition with Convolutional Neural Networks (III)"

In the articles, we introduced how to solve image classification problems using PyTorch. Another important problem in machine learning is sequence prediction. Unlike image classification, sequence prediction requires considering the correlation between data. Recurrent Neural Network (RNN) is a neural network that can handle sequence data, as it saves the previous data information at each time step. In this article, we will use a simple RNN to predict the sine function.

There is an example in the PyTorch official repository <https://github.com/pytorch/examples/>, which uses Long Short-Term Memory (LSTM) to predict the sine function. We will not consider LSTM for now, but use a simple RNN to predict the sine function.

The code for this article can be found in the `T04_series_rnn` folder in my GitHub repository <https://github.com/jin-li/pytorch-tutorial>.

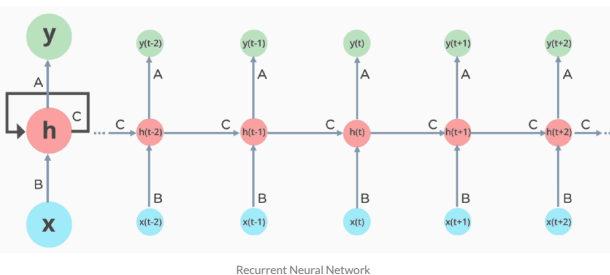
Recurrent Neural Network (RNN) Introduction

In the previous articles, we introduced feedforward neural networks, including fully connected neural networks and convolutional neural networks. Both of them are feedforward neural networks. A disadvantage of feedforward neural networks is that they cannot handle sequence data because they do not store previous data information. Recurrent Neural Network (RNN) is a neural network that can handle sequence data. It saves the previous data information at each time step, so it can handle sequence data. It is worth noting that the sequence data here is not necessarily time series data, but can also be spatial sequence data. For example, sentences in natural language processing are spatial sequence data, and audio data in speech recognition is sequence data. For convenience, we call each element in the sequence a time step.

The structure of RNN is not complicated. Its core has two points:

- During training, the training data needs to be unfolded according to time steps, and then the loss function is calculated by traversing each time step. Finally, the weights are updated through the backpropagation algorithm.
- For each time step, there is not only an output but also a hidden state. This hidden state is passed to the next time step, thus retaining the previous data information.

For more information about RNN, you can refer to the CS230 course slides from Stanford University. Here we use an animated image to briefly illustrate how RNN works:



Here, x is the input, h is the hidden state, y is the output, and t is the time step. It can be seen that the hidden state h is passed to the next time step at each time step, thus retaining the previous data information. x , h , and y are each a layer of the neural network. h is the hidden layer, x is the input layer, and y is the output layer. The size of each layer can be determined according to the specific problem.

Sine Function Prediction



Jin Li

Fate lies within the light cone.



Home

About

Archives

Search

Links

The sine function can be regarded as a time series. At some moments, the value of the sine function may be the same, but the values after that may be different. For example, for $y = \sin(x)$, when $x = 0$ and $x = \pi$, y is both 0, but at the next time step of these two moments (assuming the time step size is 0.01, then the next two time steps are $x = 0.01$ and $x = \pi + 0.01$), y is different. To predict the value of the next time step, we need to know not only the value of the current time step but also the values of the previous several time steps. This is exactly the purpose of the Recurrent Neural Network.

RNN Model Design

This problem is relatively simple, and we only need to use a Recurrent Neural Network. The input of our Recurrent Neural Network is a sequence, and the output is the next value of this sequence. The input sequence is the value of the sine function, and the output is the next value of the sine function. The structure of our Recurrent Neural Network is as follows:

- Input layer: The size of the input layer is 1, that is, there is only one input at each time step.
- Hidden layer: The size of the hidden layer can be chosen arbitrarily. Considering that this problem is relatively simple, we choose 10 neurons as the hidden layer.
- Output layer: The size of the output layer is 1, that is, there is only one output at each time step.

In this way, the structure of our Recurrent Neural Network is determined. We can use the `nn.RNN` class in PyTorch to implement this Recurrent Neural Network:

```
1 class SimpleRNN(nn.Module):
2
3     def __init__(self, rnn_type, input_size, hidden_size, num_layers):
4         super(SimpleRNN, self).__init__()
5
6         self.hidden_size = hidden_size
7         self.num_layers = num_layers
8
9         self.rnn = nn.RNN(input_size=input_size, hidden_size=hidden_size, dr
10        self.out = nn.Linear(hidden_size, 1) # Linear layer is output of m
11
12     def forward(self, x, h_state):
13         # Define our forward pass, we take some input sequence and an initi
14         r_out, h_state = self.rnn(x, h_state)
15         final_y = self.out(r_out[:, -1, :]) # Return only the last output c
16
17     return final_y, h_state
```

Here we define a `SimpleRNN` class, which inherits from PyTorch's `nn.Module` class. In the `__init__` function, we define an instance of the `nn.RNN` class, which is our Recurrent Neural Network. In the `forward` function, we define the forward propagation process of the Recurrent Neural Network, that is, how we calculate the output. Here we only need to return the output of the last time step.

Data Preparation

In the previous three articles, we used datasets that others had prepared when training neural networks. But here, we need to prepare the data ourselves. For this problem, data preparation is very simple. We only need to generate some sine function data. After generating the data, we need to encapsulate it into a PyTorch `Dataset` class so that we can easily load the data using PyTorch's `DataLoader` class.

The complete code for generating the dataset can be found in the `SineWaveDataset.py` file in the `T04_series_rnn` folder in the GitHub repository corresponding to this article <https://github.com/jin-li/pytorch-tutorial>. The specific code is as follows:

```
1 import numpy as np
2 import torch
3 from torch.utils.data import Dataset
4
5 class RNNDataset(Dataset):
6
7     def __init__(self, x, y=None):
8         self.data = x
9         self.labels = y
10
11     def __len__(self):
12         return self.data.shape[0]
13
14     def __getitem__(self, idx):
15         if self.labels is not None:
16             return self.data[idx], self.labels[idx]
17         else:
18             return self.data[idx]
19
20
21 def create_dataset(sequence_length, train_percent=0.8):
22
23     # Create sin wave at discrete time steps.
24     num_time_steps = 2000
25     time_steps = np.linspace(start=0, stop=1000, num=num_time_steps, dtype=r
26     discrete_sin_wave = (np.sin(time_steps * 2 * np.pi / 20)).reshape(-1, 1'
27
28     # Take (sequence_length + 1) elements & put as a row in sequence_data, e
29     # Move one time step and keep grabbing till we reach the end of our samp
30     sequence_data = []
31     for i in range(num_time_steps - sequence_length):
32         sequence_data.append(discrete_sin_wave[i: i + sequence_length + 1, ])
33     sequence_data = np.array(sequence_data)
34
35     # Split for train/val.
36     num_total_samples = sequence_data.shape[0]
```

```

37 num_train_samples = int(train_percent * num_total_samples)
38
39 train_set = sequence_data[:num_train_samples, :]
40 test_set = sequence_data[num_train_samples:, :]
41
42 print('{} total sequence samples, {} used for training'.format(num_total
43
44 # Take off the last element of each row and this will be our target val
45 x_train = train_set[:, :-1][:, :, np.newaxis]
46 y_train = train_set[:, -1][:, np.newaxis]
47 x_test = test_set[:, :-1][:, :, np.newaxis]
48 y_test = test_set[:, -1][:, np.newaxis]
49
50 train_data = RNNDataset(x_train, y_train)
51 test_data = RNNDataset(x_test, y_test)
52
53 torch.save(train_data, 'train_data.pt')
54 torch.save(test_data, 'test_data.pt')
55
56 if __name__ == '__main__':
57     create_dataset(sequence_length=80)

```

1. First, we define an `RNNDataset` class to store the training data. It inherits from PyTorch's `Dataset` class so that we can use PyTorch's `DataLoader` class to load the data.
2. Then we define a `create_dataset` function to generate the sine function data. This function has two input parameters. One is `sequence_length`, which indicates how many time steps of data we use to predict the next time step of data. The other is `train_percent`, which indicates how much of the data we use for training, and the rest is used for testing. The main work of this function is to generate the sine function data and encapsulate it into an instance of the `RNNDataset` class. Finally, we save the training data and test data to the files `train_data.pt` and `test_data.pt`.
3. In the `create_dataset` function, we first generate 2000 time steps of sine function data. Then we generate some sequence data for training from this sine data. The method of generating training data is:
 - o Starting from the first time step, take 50 consecutive time steps of data as a sequence, i.e., $x_1, x_2, \dots, x_{80}$.
 - o The next time step after these 50 time steps is the value to be predicted, $y = x_{81}$.
 - o Repeat the above process until all time steps are taken. Here we have 2000 time steps, so we can generate a total of $2000 - 80 = 1920$ sequences.
4. We divide these 1920 sequence data into two parts, 80% for training ($1920 \times 0.8 = 1536$) and 20% for testing.
5. Finally, we encapsulate the training data and test data into instances of the `RNNDataset` class and save them to the files `train_data.pt` and `test_data.pt`.
 - o `x_train` is a 3-D tensor with a size of $1536 \times 80 \times 1$, where 1536 is the number of training samples, 80 is the sequence length, and 1 is the input size.
 - o `y_train` is a 2-D tensor with a size of 1536×1 , where 1536 is the number of training samples, and 1 is the output size.

Model Training

As in the previous articles, we need to define a training function to train our model. The code for this training function is also simple:

```

1 def train(model, device, train_dataloader, loss_function, optimizer, epoch_i
2     model.train()
3     trained_cnt = 0
4     for batch_idx, (x_batch, y_batch) in enumerate(train_dataloader):
5         x_batch, y_batch = x_batch.to(device), y_batch.to(device)
6         h_state = torch.zeros([model.num_layers, x_batch.size()[0], model.hi
7
8         optimizer.zero_grad()
9         output, _ = model(x_batch, h_state)
10        loss = loss_function(output, y_batch)
11        loss.backward()
12        optimizer.step()
13
14        trained_cnt += len(x_batch)
15        if batch_idx % log_interval == 0:
16            print('Train Epoch: {:5d} [{:5d} / {:5d}] ({:3.0f}%)\tLoss: {:.:
17                  epoch_idx, trained_cnt, len(train_dataloader.dataset),
18                  100. * (batch_idx + 1) / len(train_dataloader), loss.it

```

The input parameters of this training function are similar to the training function in the previous articles, namely the model, training data loader, loss function, and optimizer, which are not repeated here.

Model Testing

In addition to the training function, we define a testing function. After training an epoch, we need to test the performance of our model on the test data. The code for the testing function is as follows:

```

1 def test(model, device, test_dataloader, loss_function):
2     model.eval()
3     loss_all = []
4
5     for x_batch, y_batch in test_dataloader:
6         x_batch, y_batch = x_batch.to(device), y_batch.to(device)
7         h_state = torch.zeros([model.num_layers, x_batch.size()[0], model.hi
8
9         output, _ = model(x_batch, h_state)
10        loss = loss_function(output, y_batch)
11        loss_all.append(loss.cpu().data.numpy())
12

```

```
13     print('Test loss: ', np.mean(loss_all))
14     return np.mean(loss_all)
```

Model Prediction

Finally, we define a prediction function to test whether our model can predict the values of a sine function sequence based on a given sequence of data. The code for the prediction function is as follows:

```
1 def predict(model, device, dataloader, prediction_steps):
2     model.eval()
3     h_state = torch.zeros([model.num_layers, 1, model.hidden_size]).to(device)
4
5     initial_input = next(iter(dataloader))[1].to(device) # Grab one initial
6     if initial_input.dim() == 2:
7         initial_input = initial_input.unsqueeze(0)
8     initial = initial_input.squeeze().cpu().numpy().tolist()
9
10    predictions = []
11    for _ in range(prediction_steps): # Predict prediction_steps steps ahead
12        pred, h_state = model(initial_input, h_state)
13        predictions.append(pred.item())
14        initial_input = pred.unsqueeze(0) # Ensure pred has the same dimension
15
16    return initial, predictions
```

Run the Model

We integrate the above code into one file and call the above functions in the `main()` function.

As before, we add some command-line parameters to control the training and testing of the model. The complete code can be found in the `T04_series_rnn` folder in my GitHub repository <https://github.com/jin-li/pytorch-tutorial>, in the `time_series_rnn.py` file.

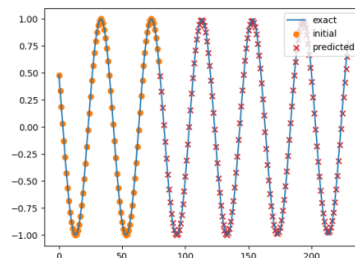
1. First, we generate the sine function dataset:

```
1 python SineWaveDataset.py
```

2. Then we train the model, use the model to predict a sine function sequence, and plot the prediction results with the true results:

```
1 python time_series_rnn.py --plot
```

This code runs for about 20 seconds on the GPU, with a memory usage of about 208M; if using the CPU, the running time increases to about 1 minute and 30 seconds. After running this command, we can see the model prediction results as shown in the figure below:



The first 80 data points are an initial sequence, and the next 150 data points are the model's prediction results. It can be seen that the model's prediction results are very close to the true results.

Summary

In this article, we introduced how to use PyTorch to implement a simple Recurrent Neural Network (RNN) to predict a sine function sequence.

In this article, I ran the example with different random seeds multiple times. The results shown above are from one of the runs with a random seed of 18 (the default value in the code on GitHub). Readers can adjust the model parameters according to the parameter selection method we introduced in the second article of this series "Learn PyTorch by Example (2): Parameter Selection in MNIST Handwritten Digit Recognition (II)" to see if they can get better results.

With this simple example as a foundation, we will introduce how to use other neural networks, such as Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), etc., to predict the sine function sequence in the next article.

PyTorch

Deep Learning

Time Series

Sequence Prediction

Machine Learning

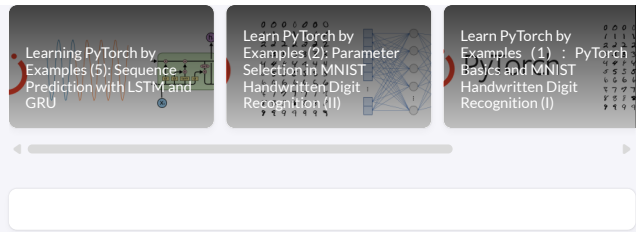
Neural Network

Recurrent Neural Network

RNN

© LICENSED UNDER CC BY-NC-SA 4.0

🕒 LAST UPDATED ON SEP 04, 2024 00:00 UTC



© 2021 - 2024 Jin Li

Built with Hugo

Theme Stack designed by Jimmy | almeida-cv designed by Almeida